MIS 600 · Database Management Systems

Chapter 4

Logical Database Design and the Relational Model

Dr. Nawaf Aljohani · W. Frank Barton School of Business · Wichita State University
Fall 2026

Figures from Hoffer, Topi, Venkataraman, & Bala (2026), Modern Database Management, 14th ed., Pearson

Where we are in the database development process

Conceptual design

Chapters 2 and 3. Understand the organization. Draw the E-R and EER diagrams. No technology yet.

Logical design

Chapter 4, tonight. Turn the diagram into relations a relational DBMS can store, with keys and integrity rules.

Physical design

Chapter 8. Decide storage, indexes, and performance. Sometimes deliberately undo a little normalization.

Four jobs tonight

1. Learn the vocabulary of the relational model: relations, keys, and the three integrity rules.
2. Transform an EER diagram into relations, seven steps, one for each kind of thing on a diagram.
3. Normalize relations so that inserting, changing, and deleting rows never produces errors.
4. Merge relations built by different teams, and see why some designers give every row an enterprise key.

An EER diagram is not a relational model. It does not have to follow normalization rules, because it was built for understanding, not for storage. Tonight is where the structuring happens.

1

The relational model

Relations, attributes, and rows. Primary, composite, and foreign keys. The properties every relation must have. A schema and an instance.

Anatomy of a relation

A relation is a named, two-dimensional table of data

EMPLOYEE1

EmpID	Name	DeptName	Salary
100	Margaret Simpson	Marketing	78,000
140	Raja Usman	Accounting	82,000
110	Chris Lucero	Info Systems	73,000
190	Lorenzo Davis	Finance	85,000
150	Deepika Priti	Marketing	72,000

After Figure 4-1, EMPLOYEE1 relation with sample data (Hoffer et al., 2026).

```
EMPLOYEE1(EmpID, Name, DeptName, Salary)
```

Shorthand notation. The underline under EmpID in the book marks the primary key.

- **Relation name:** EMPLOYEE1. It refers to the design, the columns and the rules, not to any particular set of rows.
- **Attribute:** a named column, the same meaning as in Chapter 2.
- **Row:** the attribute values for one entity. Here, one employee.
- **Primary key:** an attribute, or combination of attributes, that uniquely identifies each row. The book underlines it.
- **Instance:** the rows you see, a snapshot of the relation's contents at one moment. Add a row, change a salary, delete every row: still the same relation.

Relational keys

- **Primary key.** An attribute, or combination of attributes, that uniquely identifies each row. Underlined.
- **Composite key.** A primary key made of more than one attribute. ORDERLINE uses OrderID plus ProductID.
- **Foreign key.** An attribute in one relation that is the primary key of another. It is the only way a relational database represents a relationship between two tables. Dashed underline in the book.

Every DeptName in EMPLOYEE1 must appear as a DeptName in DEPARTMENT. Marketing appears twice on the left and once on the right, and the match is the relationship.

EMPLOYEE1

EmpID	Name	DeptName (FK)	Salary
100	Margaret Simpson	Marketing	78,000
140	Raja Usman	Accounting	82,000
110	Chris Lucero	Info Systems	73,000
190	Lorenzo Davis	Finance	85,000
150	Deepika Priti	Marketing	72,000

DEPARTMENT

DeptName	Location	Fax
Marketing	Woolsey 210	316-978-3001
Accounting	Woolsey 305	316-978-3002
Info Systems	Woolsey 308	316-978-3003
Finance	Woolsey 320	316-978-3004

Not every table is a relation

Six properties a table must have. The table on the right breaks one of them.

Property	What it rules out
Each relation has a unique name	Two tables both called CUSTOMER
Every cell holds one value (atomic)	A cell that holds a list. The table on the right fails here.
No two rows are identical	Entering the same employee twice
Each column has a unique name	Two columns both called Address
Column order does not matter	Any meaning that depends on which column comes first
Row order does not matter	Any meaning that depends on row order

Employee data with courses taken

EmpID	Name	Dept	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	78,000	SPSS Surveys	6/19/2024 10/7/2024
140	Raja Usman	Accounting	82,000	Tax Acc	12/8/2024
110	Chris Lucero	Info Systems	73,000	Visual Basic C++	1/12/2024 4/22/2024
190	Lorenzo Davis	Finance	85,000		
150	Deepika Priti	Marketing	72,000	SPSS Java	6/19/2024 8/12/2024

After Figure 4-2(a), a table with repeating groups (Hoffer et al., 2026). The book's learning objective says "five properties" and the chapter lists six: the two order rules are often counted as one.

Removing multivalued attributes

- The fix is simple: fill the empty cells so that every course gets a complete row of its own.
- EMPLOYEE2 now satisfies the atomic property. It is a relation.
- But EmpID no longer identifies a row. Employee 100 appears twice. The primary key has to be EmpID plus CourseTitle.
- And every employee's name, department, and salary are repeated once per course. That redundancy is the problem normalization fixes later tonight.

EmpID	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	78,000	SPSS	6/19/2024
				Surveys	10/7/2024
140	Raja Usman	Accounting	82,000	Tax Acc	12/8/2024
110	Chris Lucero	Info Systems	73,000	Visual Basic	1/12/2024
				C++	4/22/2024
190	Lorenzo Davis	Finance	85,000		
150	Deepika Priti	Marketing	72,000	SPSS	6/16/2024
				Java	8/12/2024

(a) Table with repeating groups

EMPLOYEE2

EmpID	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	78,000	SPSS	6/19/2024
100	Margaret Simpson	Marketing	78,000	Surveys	10/7/2024
140	Raja Usman	Accounting	82,000	Tax Acc	12/8/2024
110	Chris Lucero	Info Systems	73,000	Visual Basic	1/12/2024
110	Chris Lucero	Info Systems	73,000	C++	4/22/2024
190	Lorenzo Davis	Finance	85,000		
150	Deepika Priti	Marketing	72,000	SPSS	6/19/2024
150	Deepika Priti	Marketing	72,000	Java	8/12/2024

(b) EMPLOYEE2 relation

Figure 4-2. Eliminating multivalued attributes (Hoffer et al., 2026)

A schema, two ways

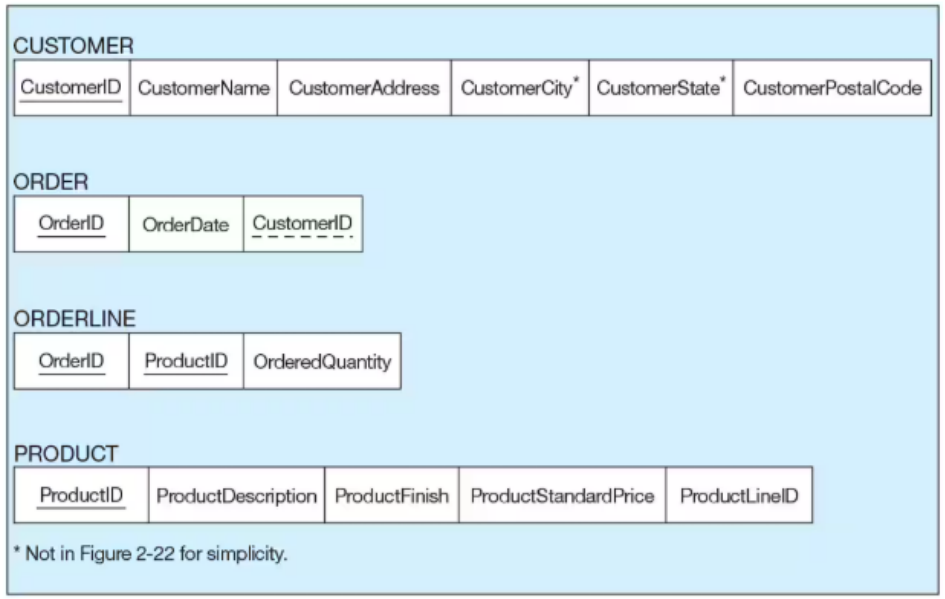


Figure 4-3. Schema for four relations, Pine Valley Furniture Company (Hoffer et al., 2026)

A schema describes the overall logical structure of the database. The graphical form shows where foreign keys point. The text form is quicker to write:

```
CUSTOMER(CustomerID, CustomerName, CustomerAddress,
         CustomerCity, CustomerState, CustomerPostalCode)
ORDER(OrderID, OrderDate, CustomerID)
ORDERLINE(OrderID, ProductID, OrderedQuantity)
PRODUCT(ProductID, ProductDescription, ProductFinish,
        ProductStandardPrice, ProductLineID)
```

- ORDERLINE has a composite primary key, OrderID plus ProductID.
- CustomerID is a foreign key in ORDER. It links each order to its customer.
- ORDERLINE has two foreign keys, and both are also parts of its primary key. That shape is how a many-to-many relationship is stored.

An instance of the schema

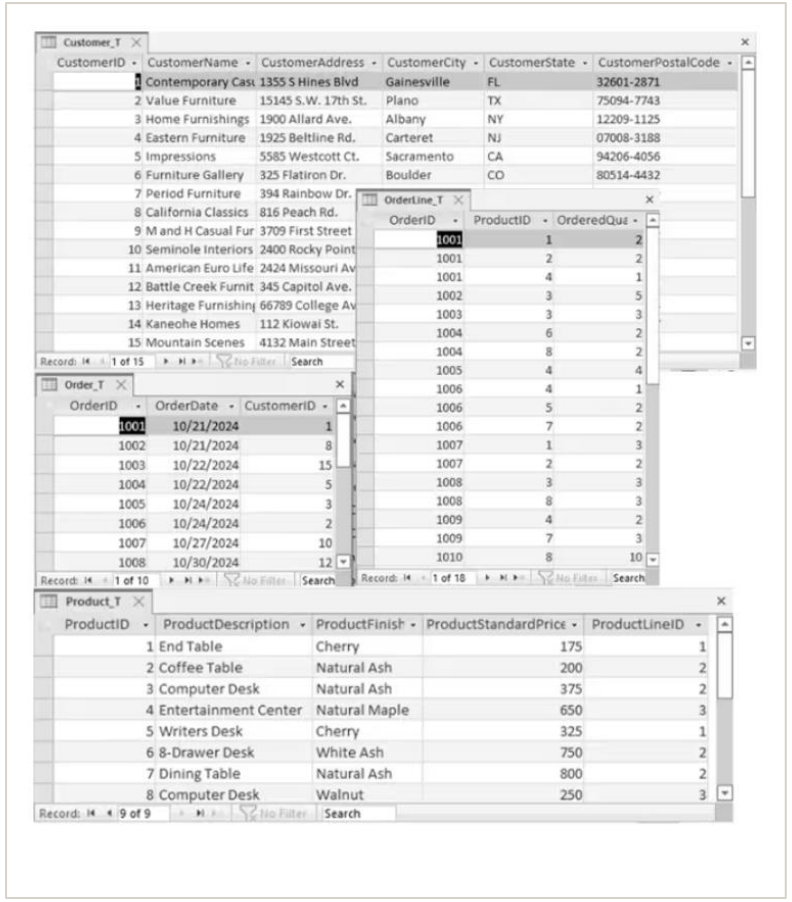


Figure 4-4. Instance of a relational schema (Hoffer et al., 2026)

The rows are not part of the relation. They are one instance, a snapshot of its contents. The book wants an instance for every design: sample data test your assumptions, check accuracy, help users react to the design, and feed prototypes.

Follow order 1006 through the tables

1. Order_T: order 1006, placed 10/24/2024, CustomerID 2.
2. Customer_T: customer 2 is Value Furniture, Plano, TX.
3. OrderLine_T: three lines, products 4 (qty 1), 5 (qty 2), and 7 (qty 2).
4. Product_T: Entertainment Center $650, Writers Desk $325, Dining Table $800.
5. Total: $650 + $650 + $1,600 = $2,900. Remember that invoice. We normalize it later tonight.

2

Integrity constraints

Domain constraints, entity integrity, and referential integrity. What a null is. What happens when you delete a row that other rows point to. The SQL that enforces all of it.

Domain constraints

- **Domain:** the set of values that may be assigned to an attribute. All values in a column must come from the same domain.
- A domain definition has five parts: name, meaning, data type, size or length, and allowable values or range.
- CustomerState draws on the domain States: character, size 2. "TX" is valid, "Texas" is not.
- When the relations become SQL tables, every data type and length comes straight from this table.

A domain is the dropdown list or the input mask on a web form. The form enforces it before the data leave the browser. The DBMS enforces it again on arrival.

Attribute	Domain Name	Description	Domain
CustomerID	Customer IDs	Set of all possible customer IDs	character: size 5
CustomerName	Customer Names	Set of all possible customer names	character: size 25
CustomerAddress	Customer Addresses	Set of all possible customer addresses	character: size 30
CustomerCity	Cities	Set of all possible cities	character: size 20
CustomerState	States	Set of all possible states	character: size 2
CustomerPostalCode	Postal Codes	Set of all possible postal zip codes	character: size 10
OrderID	Order IDs	Set of all possible order IDs	character: size 5
OrderDate	Order Dates	Set of all possible order dates	date: format mm/dd/yy
ProductID	Product IDs	Set of all possible product IDs	character: size 5
ProductDescription	Product Descriptions	Set of all possible product descriptions	character: size 25
ProductFinish	Product Finishes	Set of all possible product finishes	character: size 15
ProductStandardPrice	Unit Prices	Set of all possible unit prices	monetary: 6 digits
ProductLineID	Product Line IDs	Set of all possible product line IDs	integer: 3 digits
OrderedQuantity	Quantities	Set of all possible ordered quantities	integer: 3 digits

Table 4-1. Domain definitions for INVOICE attributes (Hoffer et al., 2026)

Entity integrity and nulls

What a null is

- **Null:** a marker that says no value applies, or the value is unknown.
- It is the absence of a value. It is not zero and it is not a string of blanks.
- Two reasons a cell is null: nothing applies (you have no landline), or the value is not known yet (you cannot recall the number).
- Nulls can make queries behave strangely, but Codd himself argued for them.

Entity integrity rule

No primary key attribute, or component of a primary key attribute, may be null.

- If the key could be null, you could never find the row again.
- The rule covers every component of a composite key. An ORDERLINE row with an OrderID and a missing ProductID has no complete key.
- SQL enforces it with NOT NULL on every primary key column plus the PRIMARY KEY constraint.

Referential integrity

- **Referential integrity constraint:** each foreign key value must match a primary key value in the referenced relation, or be null.
- Before you insert an order, its customer must already exist in CUSTOMER.
- In the graphical schema, every arrow from a foreign key to a primary key is one constraint the DBMS must enforce. Two of the three arrows leave ORDERLINE.
- Can a foreign key be null? If every order must have a customer (mandatory), no. If the relationship is optional, yes. Decide it when you define the table.

A foreign key is a pointer. Referential integrity says a pointer must never dangle.

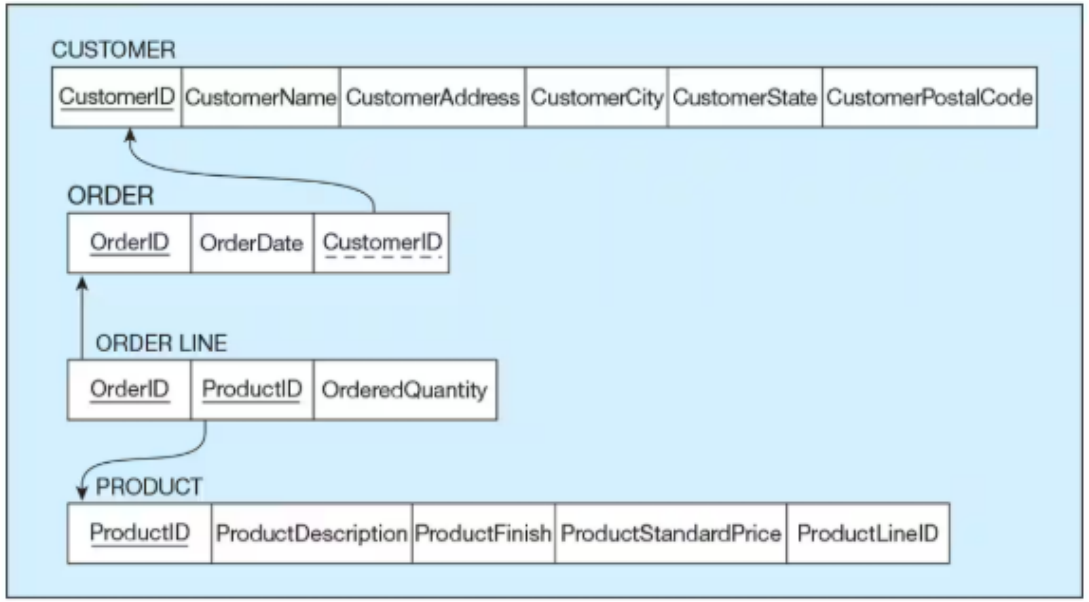


Figure 4-5. Referential integrity constraints (Hoffer et al., 2026)

Try to break the rules

Order_T(OrderID, OrderDate, CustomerID) as defined in Figure 4-6: OrderID is NOT NULL, CustomerID references Customer_T

Attempted INSERT into Order_T	What the DBMS does	Which rule
OrderID 1011, CustomerID 2	Accepted. Customer 2 exists.	All satisfied
OrderID 1011, CustomerID 99	Refused. No customer 99 in Customer_T: constraint Order_FK violated.	Referential integrity
OrderID NULL, CustomerID 2	Refused. A primary key column cannot be null.	Entity integrity
OrderID 1006, CustomerID 2	Refused. Order 1006 already exists: constraint Order_PK violated.	Uniqueness of the key
OrderID 1011, CustomerID NULL	Accepted. A null foreign key satisfies referential integrity. The book's Order_T does not declare CustomerID NOT NULL. Whether it should is a business rule.	Referential integrity (null allowed)

And when you delete a customer who has orders?

Cascading delete	The customer and all of that customer's orders are deleted. The sales history goes with the customer.
Prohibit (restrict)	The customer cannot be deleted until the orders are deleted first. The book calls this a safety check.
Set null	The orders survive with CustomerID set to NULL. The history stays, the link to the customer is lost.

The SQL that enforces all three rules

```
CREATE TABLE Customer_T
        (CustomerID                 NUMBER(11,0)      NOT NULL,
        CustomerName                VARCHAR2(25)      NOT NULL,
        CustomerAddress             VARCHAR2(30),
        CustomerCity                VARCHAR2(20),
        CustomerState               CHAR(2),
        CustomerPostalCode          VARCHAR2(9),
CONSTRAINT Customer_PK PRIMARY KEY (CustomerID));

CREATE TABLE Order_T
        (OrderID                    NUMBER(11,0)      NOT NULL,
        OrderDate                   DATE DEFAULT SYSDATE,
        CustomerID                  NUMBER(11,0),
CONSTRAINT Order_PK PRIMARY KEY (OrderID),
CONSTRAINT Order_FK FOREIGN KEY (CustomerID) REFERENCES Customer_T (CustomerID));

CREATE TABLE Product_T
        (ProductID                  NUMBER(11,0)      NOT NULL,
        ProductDescription          VARCHAR2(50),
        ProductFinish               VARCHAR2(20),
        ProductStandardPrice        DECIMAL(6,2),
        ProductLineID               NUMBER(11,0),
CONSTRAINT Product_PK PRIMARY KEY (ProductID));

CREATE TABLE OrderLine_T
        (OrderID                    NUMBER(11,0)      NOT NULL,
        ProductID                   NUMBER(11,0)      NOT NULL,
        OrderedQuantity             NUMBER(11,0),
CONSTRAINT OrderLine_PK PRIMARY KEY (OrderID, ProductID),
CONSTRAINT OrderLine_FK1 FOREIGN KEY (OrderID) REFERENCES Order_T (OrderID),
CONSTRAINT OrderLine_FK2 FOREIGN KEY (ProductID) REFERENCES Product_T (ProductID));
```

Figure 4-6. SQL table definitions (Hoffer et al., 2026)

Rule	Where it lives in the SQL
Domain constraints	The data types and lengths. VARCHAR2(25) is text up to 25 characters, NUMBER(11,0) an 11-digit whole number, CHAR(2) exactly two characters, DECIMAL(6,2) six digits with two after the point, DATE DEFAULT SYSDATE a date that defaults to today. Every one comes from Table 4-1.
Entity integrity	NOT NULL on every primary key column plus the PRIMARY KEY constraint. OrderLine_T shows a composite key: PRIMARY KEY (OrderID, ProductID).
Referential integrity	One FOREIGN KEY … REFERENCES clause for each arrow in Figure 4-5. Order_T has one. OrderLine_T has two.

The foreign key may take a different name from the key it references (CustNo would work), but it must share its domain. The _T suffix is the book's way of telling tables from relations.

3

Well-structured relations

Minimal redundancy, and rows you can insert, change, and delete without errors.
The three anomalies that show up when a table is about two things at once.

Three anomalies in one table

EMPLOYEE2 stores employees and their courses together

EMPLOYEE2

EmpID	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	80,000	SPSS	6/19/2024
100	Margaret Simpson	Marketing	78,000	Surveys	10/7/2024
140	Raja Usman	Accounting	82,000	Tax Acc	12/8/2024
110	Chris Lucero	Info Systems	73,000	Visual Basic	1/12/2024
110	Chris Lucero	Info Systems	73,000	C++	4/22/2024
190	Lorenzo Davis	Finance	85,000		
150	Deepika Priti	Marketing	72,000	SPSS	6/19/2024
150	Deepika Priti	Marketing	72,000	Java	8/12/2024
200	*Ana Reyes*	*Finance*	*60,000*	**NULL?**	*NULL?*

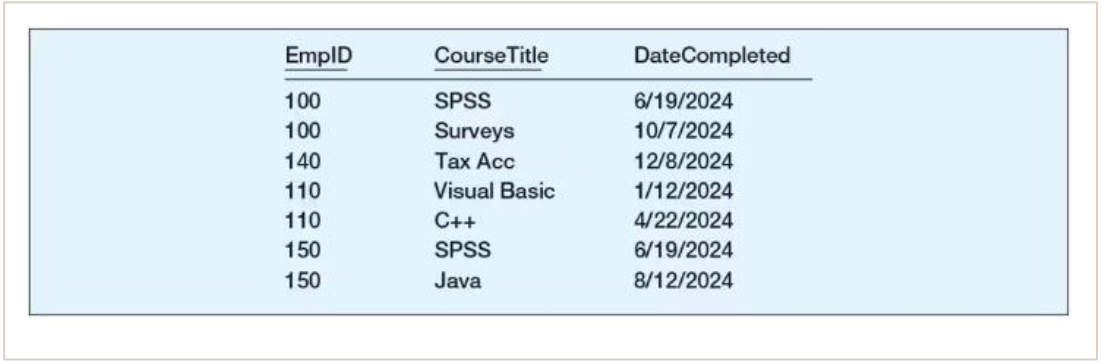
After Figure 4-2(b) (Hoffer et al., 2026).

Anomaly	What goes wrong
Insertion	Ana Reyes has not taken a course yet. CourseTitle is part of the key and cannot be null, so her row cannot be inserted.
Deletion	Delete employee 140 and the only record that Tax Acc had an offering completed on 12/8/2024 disappears too.
Modification	Employee 100 got a raise. It was written in one row and missed in the other. The table now contradicts itself.

A well-structured relation has minimal redundancy and lets users insert, modify, and delete rows without errors or inconsistencies. EMPLOYEE2 is not one, because it is about two things at once.

The fix: one relation per entity

- EMPLOYEE1 holds each employee once. EMPCOURSE holds each course completion once, keyed by EmpID plus CourseTitle.
- Insert Ana with no course. Delete employee 140 without losing Tax Acc. Change a salary in one place.
- **One fact, one place.** Every fact stored exactly once, and every table about exactly one kind of thing. That is the whole idea, and normalization is the formal way to get there.



EmpID	CourseTitle	DateCompleted
100	SPSS	6/19/2024
100	Surveys	10/7/2024
140	Tax Acc	12/8/2024
110	Visual Basic	1/12/2024
110	C++	4/22/2024
150	SPSS	6/19/2024
150	Java	8/12/2024

Figure 4-7. EMPCOURSE relation with sample data (Hoffer et al., 2026)

4

From EER diagrams to relations

Seven steps, one for each kind of thing that can appear on a diagram. Almost every step comes down to one question: where does the foreign key go?

The seven steps

Step	What it maps	Figures
1	Regular entities, composite attributes, multivalued attributes	4-8 to 4-10
2	Weak entities, and when to add a surrogate key	4-11
3	Binary relationships: 1:M, M:N, 1:1	4-12 to 4-14
4	Associative entities, with and without an identifier	4-15, 4-16
5	Unary relationships: 1:M and M:N	4-17, 4-18
6	Ternary relationships	4-19
7	Supertype/subtype relationships	4-20, 4-21

Why learn it when CASE tools can do it?

- Tools often cannot handle ternary and supertype/subtype structures.
- Some situations have several legitimate answers. You choose.
- You have to check the quality of what a tool produces.
- It shows why conceptual and logical modeling are different jobs.

A relationship on a diagram is a line. A relational database has no lines, only tables. So every line becomes either a foreign key column in an existing table, or a new table whose whole job is to hold two foreign keys.

Step 1: Map regular entities

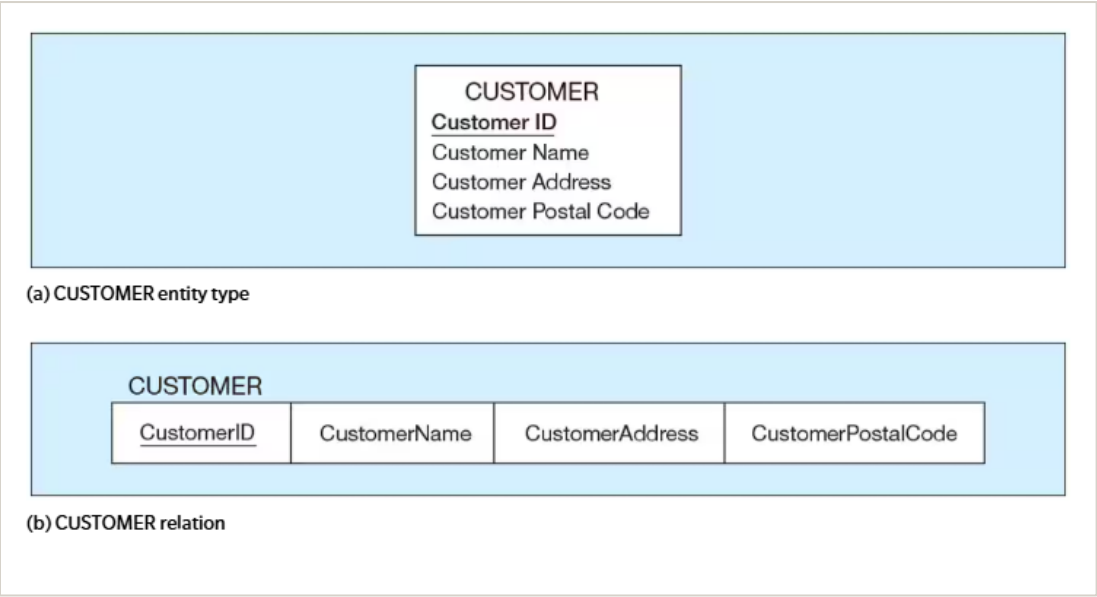


Figure 4-8. Mapping a regular entity (Hoffer et al., 2026)



Figure 4-9. Mapping a composite attribute (Hoffer et al., 2026)

- Each regular entity becomes one relation. Simple attributes become attributes. The identifier becomes the primary key.
- A composite attribute contributes only its simple components: CustomerStreet, CustomerCity, CustomerState. Stored separately, they are easier to report on and to keep clean.

Step 1: A multivalued attribute becomes its own relation

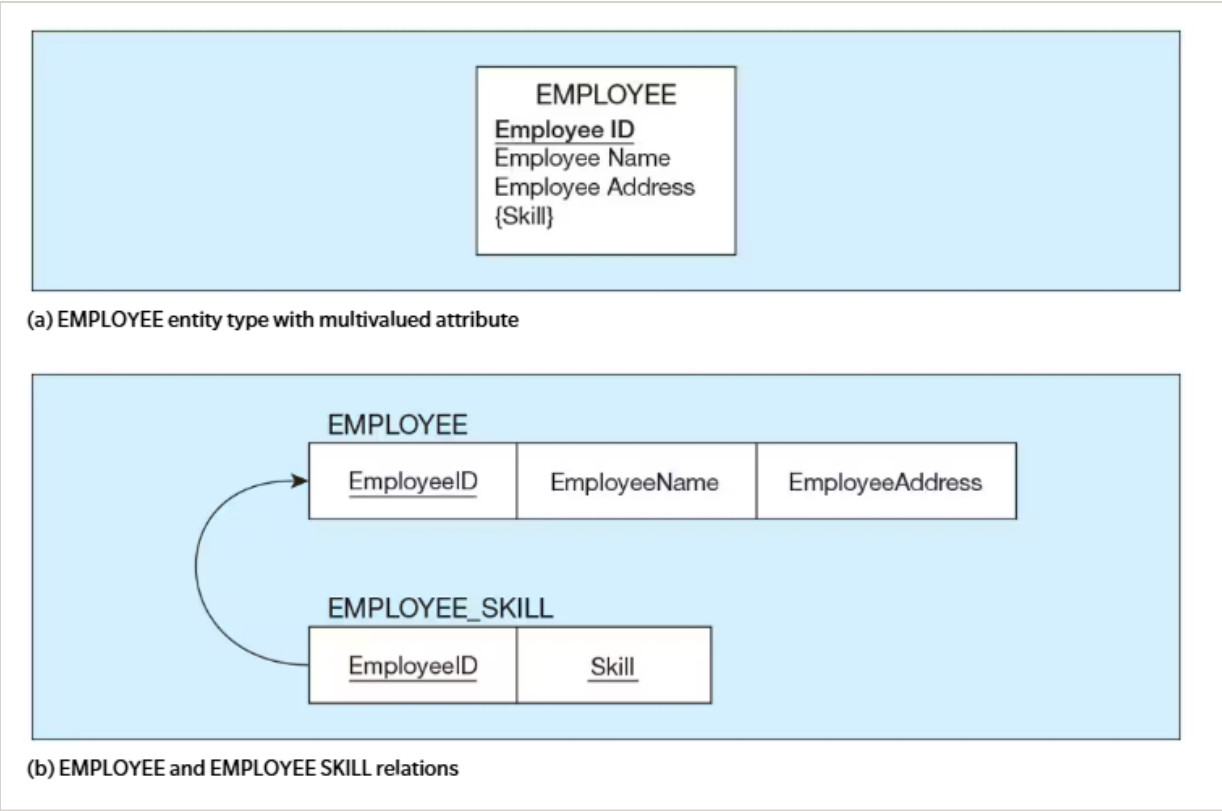


Figure 4-10. Mapping an entity with a multivalued attribute (Hoffer et al., 2026)

- Two relations instead of one. The first holds everything except the multivalued attribute.
- The second holds just two attributes: the first relation's primary key (a foreign key here) and the multivalued attribute. Together they are its primary key.
- EMPLOYEE_SKILL has no nonkey attributes. Each row records one fact: this employee has this skill. Users may want more, such as YearsExperience.
- If SKILL needs attributes of its own, make a SKILL relation. EMPLOYEE_SKILL then becomes an associative entity between the two.

A multivalued attribute is a cell that wants to hold a list. Cells cannot hold lists, so the list becomes a table with one row per item.

Step 2: Map weak entities

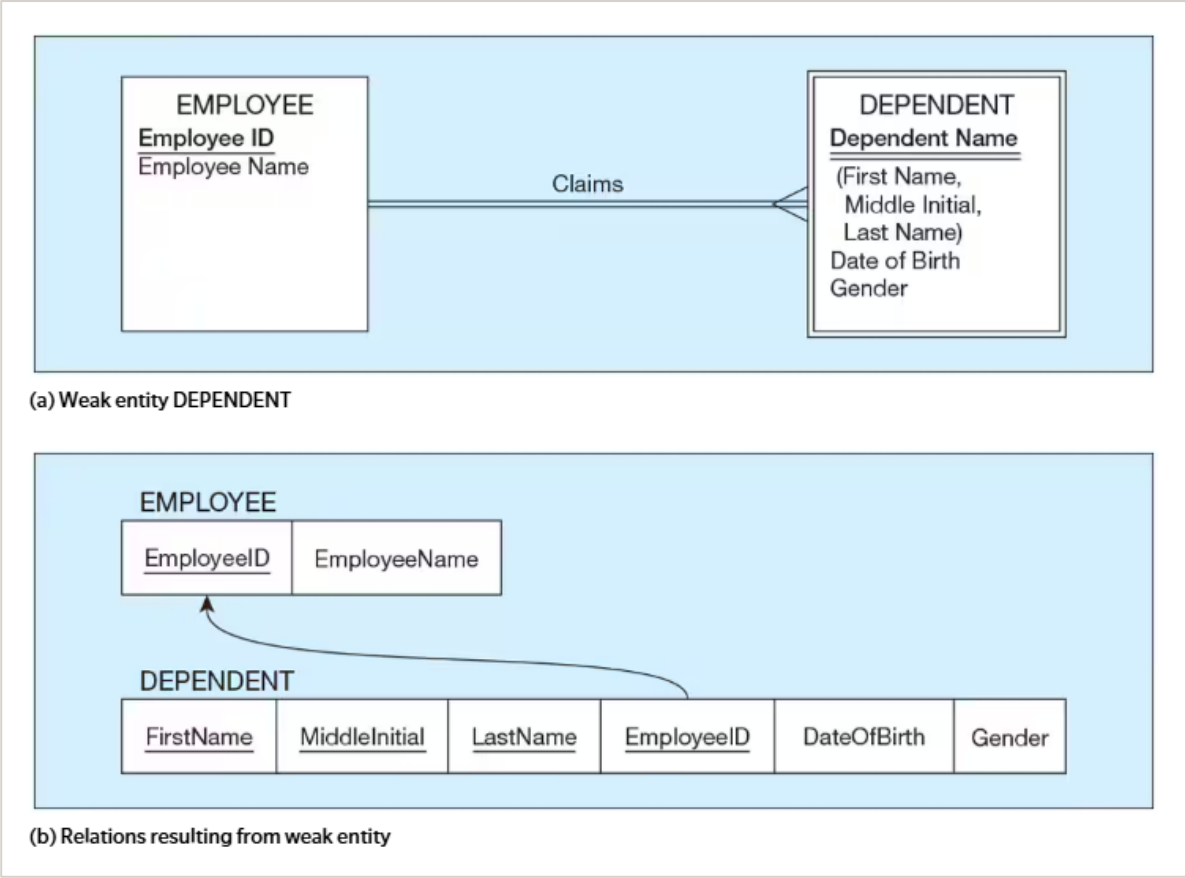


Figure 4-11. Mapping a weak entity (Hoffer et al., 2026)

- Create a relation with the weak entity's simple attributes. Add the owner's primary key as a foreign key.
- The primary key is the owner's key plus the partial identifier: EmployeeID, FirstName, MiddleInitial, LastName. Four parts.
- **Surrogate primary key:** a serial number, DependentID, that replaces the four-part key. EmployeeID stays as a foreign key.
- **Create a surrogate key when** the primary key is composite, the natural key is long and copied into every referencing table, or the natural key is recycled or cannot be guaranteed unique.

Keep the natural key in the relation as nonkey data. Users search by it and never see the surrogate.

Step 3: Binary one-to-many

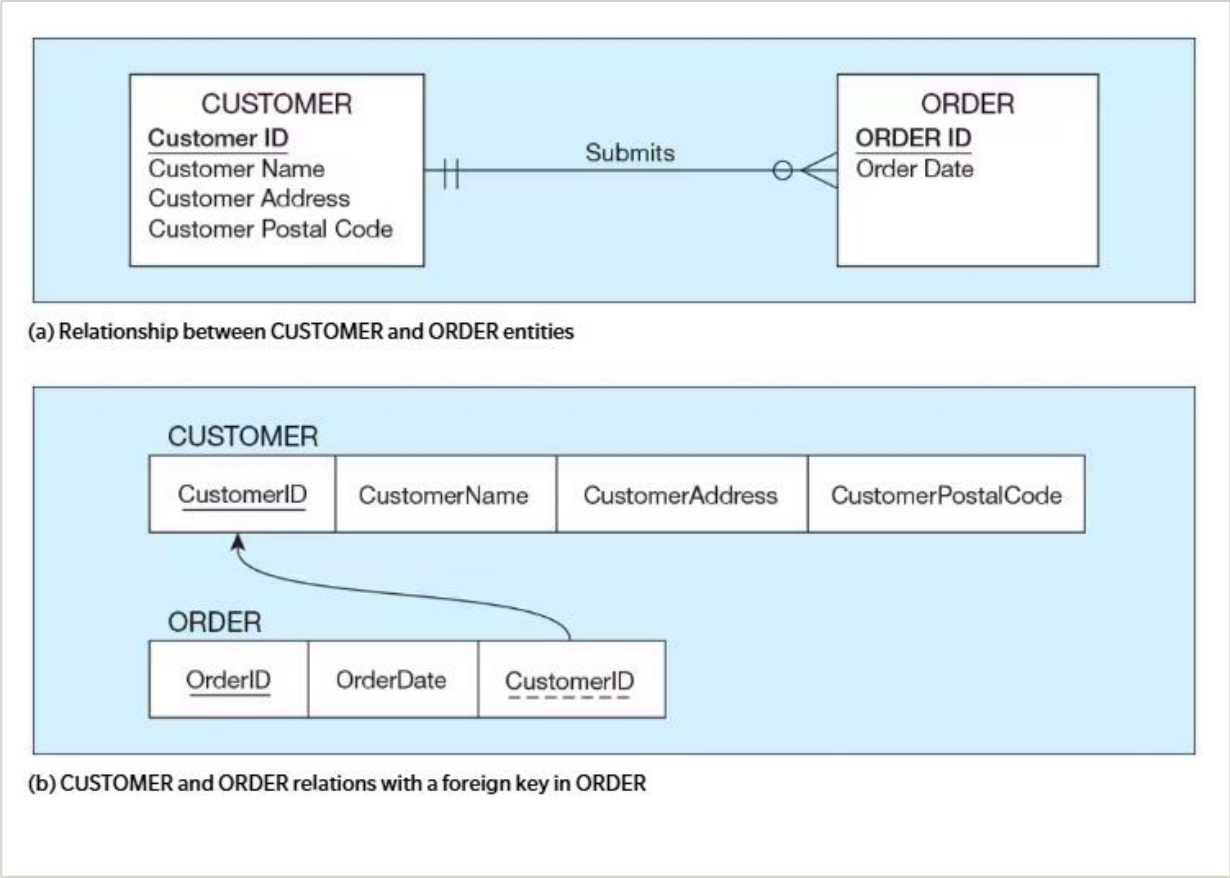


Figure 4-12. Mapping a 1:M relationship (Hoffer et al., 2026)

- Map both entities with Step 1.
- Take the primary key of the entity on the one side and put it in the relation on the many side as a foreign key.
- One customer submits many orders, so CustomerID goes into ORDER.
- The foreign key may have a different name from the primary key it references. It must have the same domain.

The book's mnemonic: the primary key migrates to the many side.

Step 3: Binary many-to-many

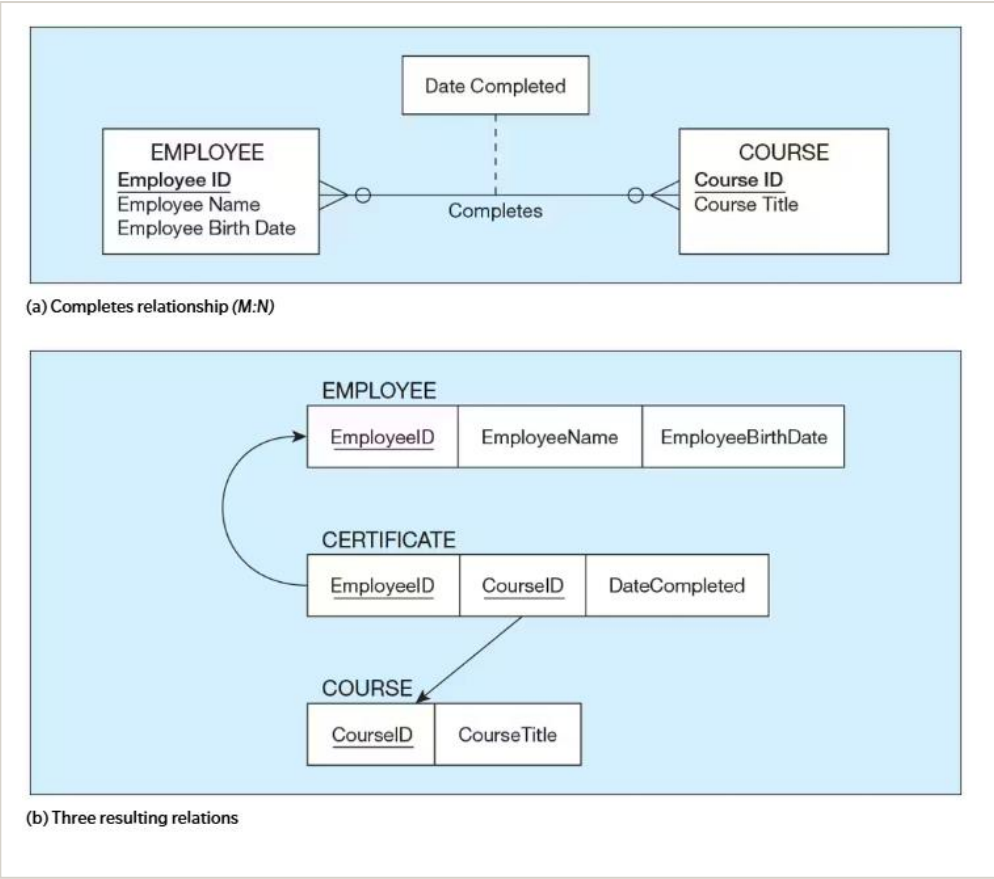


Figure 4-13. Mapping an M:N relationship (Hoffer et al., 2026)

- Neither side can hold the other's key: one cell cannot list many values.
- So create a new relation. Put both primary keys in it as foreign keys. Together they form its composite primary key.
- Attributes of the relationship itself go into the new relation: DateCompleted lives in CERTIFICATE.
- It is often wise to add a surrogate key to a relation like CERTIFICATE.

Step 3: Binary one-to-one

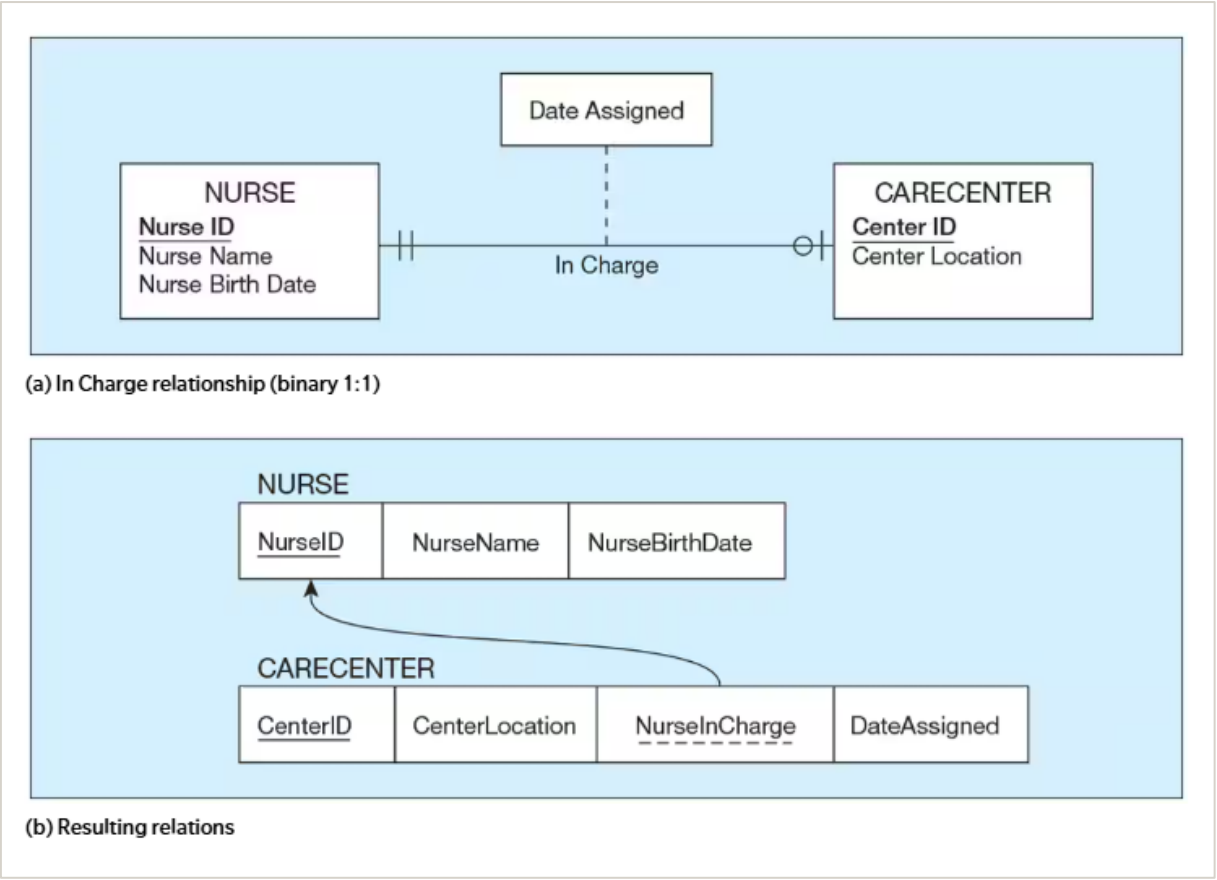


Figure 4-14. Mapping a binary 1:1 relationship (Hoffer et al., 2026)

- A special case of one-to-many. Create both relations, then put one primary key into the other relation as a foreign key.
- Which one? In a 1:1 relationship one direction is almost always mandatory and the other optional.
- Every care center must have a nurse in charge. Most nurses are not in charge of anything.
- So the foreign key goes into CARE CENTER, where it is never null. DateAssigned, an attribute of the relationship, goes with it.

Put the foreign key in the table whose rows always have a partner.

Where does the foreign key go?

Decide before the next slide

1. CUSTOMER submits ORDER

One customer, many orders. Each order has exactly one customer.

A. Put CustomerID in ORDER

B. Put OrderID in CUSTOMER

C. Create SUBMITS(CustomerID, OrderID)

2. EMPLOYEE completes COURSE

Many to many, with Date Completed on the relationship.

A. Put CourseID and DateCompleted in EMPLOYEE

B. Put EmployeeID and DateCompleted in COURSE

C. Create CERTIFICATE(EmployeeID, CourseID, DateCompleted)

3. NURSE in charge of CARE CENTER

Every care center has one nurse in charge. A nurse may or may not be in charge.

A. Put NurseInCharge in CARE CENTER

B. Put CenterID in NURSE

C. Put a foreign key in both

Where does the foreign key go?

Answers

1. CUSTOMER submits ORDER

A. CustomerID goes into ORDER.

The primary key migrates to the many side. Each order has one customer, so the value fits in one cell. OrderID in CUSTOMER would need to hold a list, and a new relation is only needed for many-to-many.

```
CUSTOMER(CustomerID, CustomerName,
CustomerAddress)
ORDER(OrderID, OrderDate, CustomerID)
```

2. EMPLOYEE completes COURSE

C. Create CERTIFICATE.

Neither side can hold the other's key. A new relation holds both keys as its composite primary key, and DateCompleted belongs to the pair.

```
EMPLOYEE(EmployeeID, EmployeeName,
EmployeeBirthDate)
CERTIFICATE(EmployeeID, CourseID,
DateCompleted)
COURSE(CourseID, CourseTitle)
```

3. NURSE in charge of CARE CENTER

A. NurseInCharge goes into CARE CENTER.

Every care center row always has a nurse to point at, so the column is never null. CenterID in NURSE would be null for most nurses, and two foreign keys store one fact twice.

```
NURSE(NurseID, NurseName, NurseBirthDate)
CARECENTER(CenterID, CenterLocation,
NurseInCharge, DateAssigned)
```

Step 4: Map associative entities

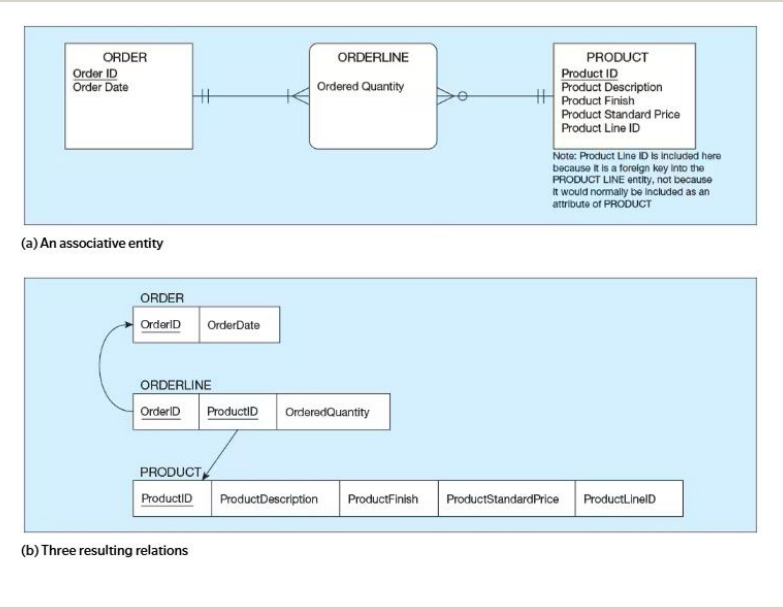


Figure 4-15. Associative entity without an identifier (Hoffer et al., 2026)

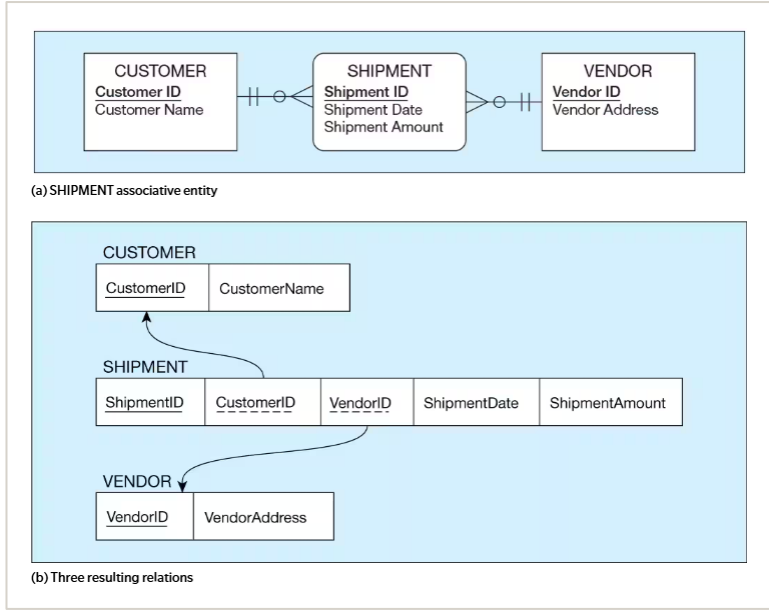


Figure 4-16. Associative entity with an identifier (Hoffer et al., 2026)

- **No identifier assigned:** the default key is the two borrowed primary keys, OrderID plus ProductID. Same result as an M:N relationship.
- **Identifier assigned:** that identifier is the key, and the two borrowed keys become plain foreign keys. ShipmentID was chosen because CustomerID plus VendorID plus Date still cannot tell two shipments apart.

Step 5: Map unary relationships

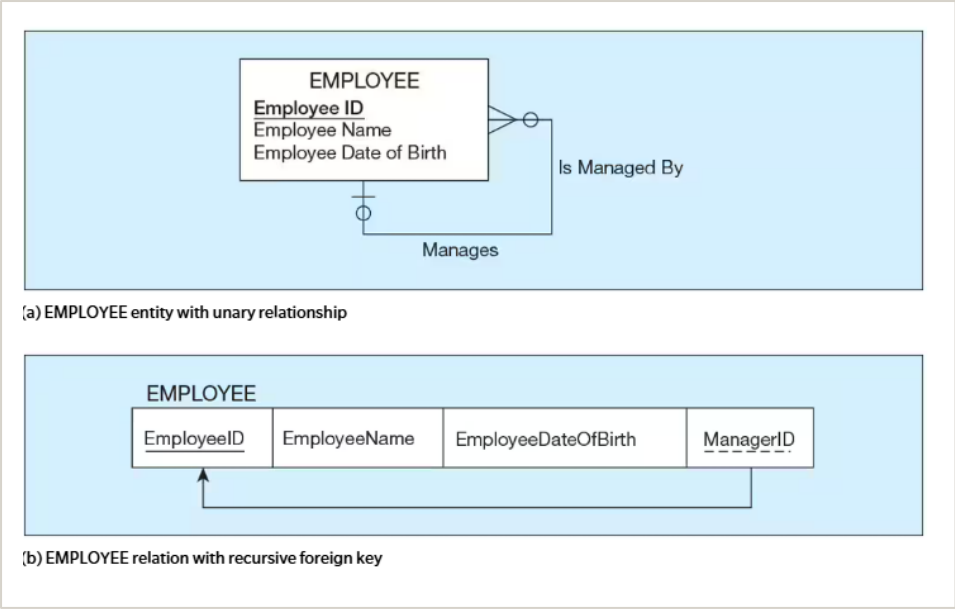


Figure 4-17. Unary 1:M, a recursive foreign key (Hoffer et al., 2026)

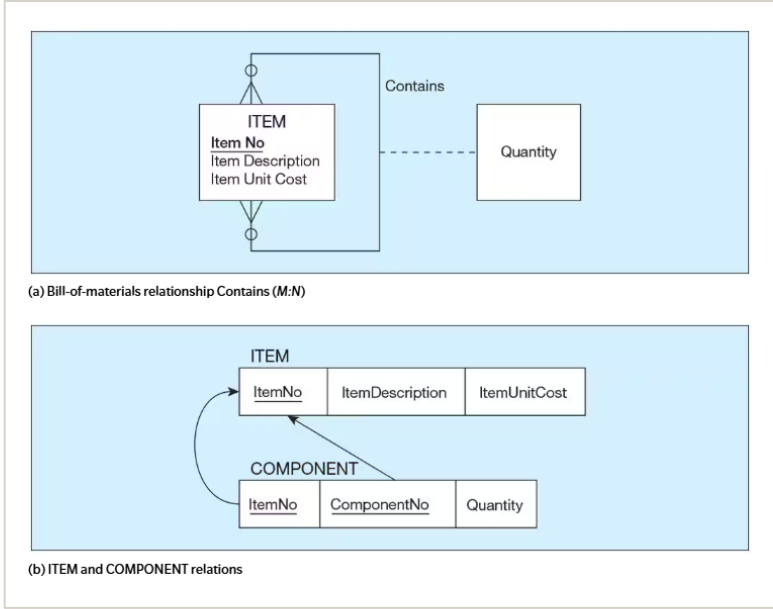


Figure 4-18. Unary M:N, an associative relation (Hoffer et al., 2026)

- **Unary 1:M:** add a foreign key to the same relation that references its own primary key. ManagerID holds the EmployeeID of the boss. An org chart in one table.
- **Unary M:N:** an associative relation whose two key attributes both come from ITEM. COMPONENT(ItemNo, ComponentNo, Quantity) is a bill of materials. SELECT ComponentNo, Quantity FROM Component_T WHERE ItemNo = 100;

Step 6: Map ternary relationships

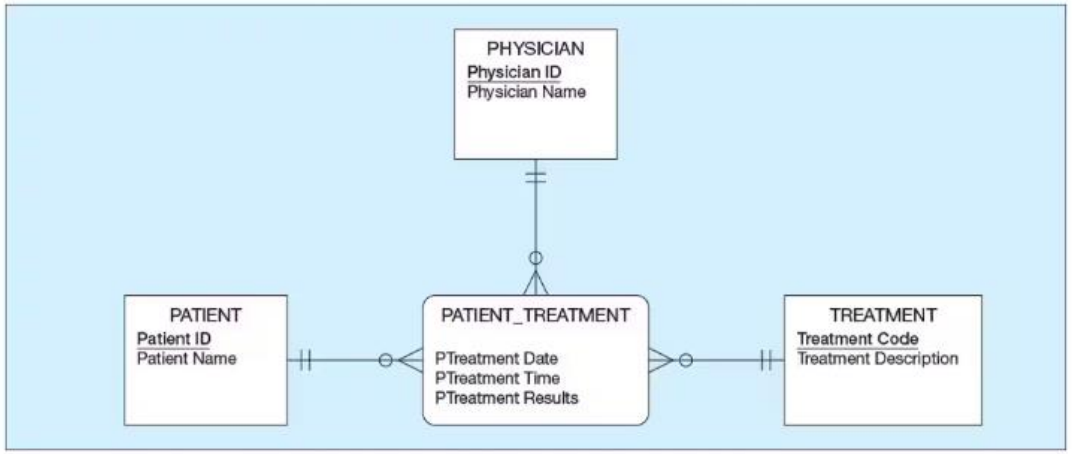


Figure 4-19. Mapping a ternary relationship (Hoffer et al., 2026)

- Create an associative relation. Its default key is the three primary keys of the participating entities, all of them also foreign keys.
- Three keys are not enough here. A patient can get the same treatment from the same physician twice.
- Adding the date is still not enough: morning and afternoon. So the key grows to five attributes.
- Better in practice: a surrogate key, PTreatmentID. Then PatientID, PhysicianID, and TreatmentCode are ordinary foreign keys.

The text calls the foreign key PTreatmentCode to show that names may differ from the key they reference. The figure labels it TreatmentCode. Same column.

Step 7: Map supertype/subtype relationships

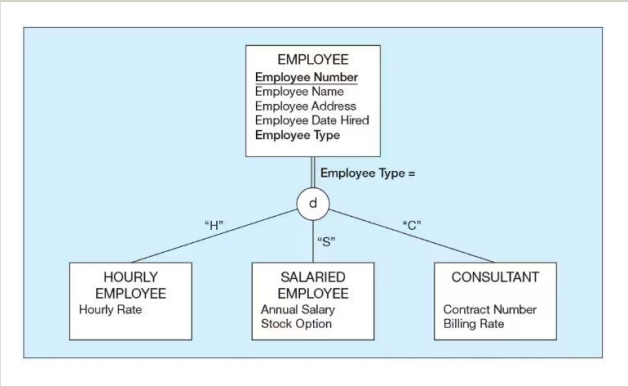


Figure 4-20 (Hoffer et al., 2026)

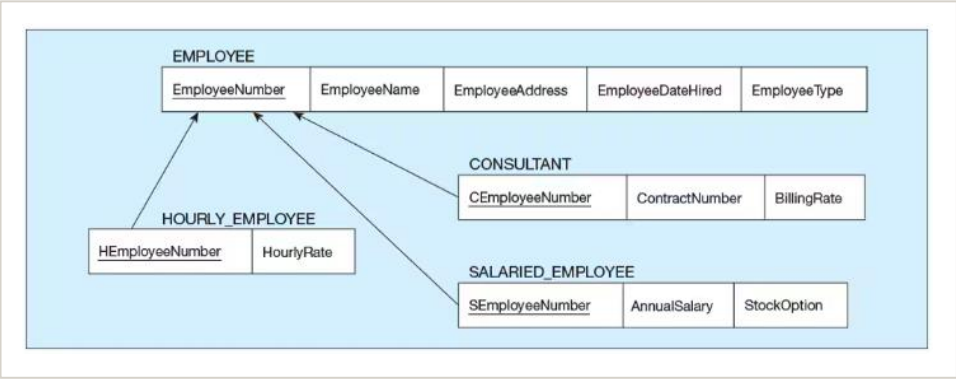


Figure 4-21. Mapping supertype/subtype relationships (Hoffer et al., 2026)

- One relation for the supertype and one for each subtype.
- The supertype relation gets the shared attributes, the primary key, and the discriminator (EmployeeType).
- Each subtype relation gets the supertype's primary key, with a prefix so the names differ, plus only its own attributes.
- Each prefixed key is a foreign key back to EMPLOYEE.
- **To see a full salaried employee, join the two:** SELECT * FROM Employee_T, SalariedEmployee_T WHERE EmployeeNumber = SEmployeeNumber;

This is the choice Chapter 3 promised: one table with nulls, or one table per subtype. The book takes one per subtype plus the supertype.

Summary of EER-to-relational transformations

After Table 4-2 (Hoffer et al., 2026)

EER structure	Relational representation	Figure
Regular entity	A relation with the primary key and the nonkey attributes.	4-8
Composite attribute	Each component becomes a separate attribute.	4-9
Multivalued attribute	A separate relation with a composite primary key that includes the entity's primary key.	4-10
Weak entity	A relation with a composite primary key (owner's key plus partial identifier) and the nonkey attributes.	4-11
Binary or unary 1:M	The one side's primary key becomes a foreign key in the many side's relation.	4-12, 4-17
Binary or unary M:N, or associative entity without its own key	A relation with a composite primary key made of the related entities' primary keys, plus any nonkey attributes.	4-13, 4-15, 4-18
Binary or unary 1:1	Either primary key in the other relation. If one side is optional, the foreign key goes in the optional side's relation.	4-14
M:N or associative entity with its own key	A relation whose primary key is the associative entity's own key, with the related entities' primary keys as foreign keys.	4-16
Ternary and n-ary	Same as M:N. All related entities' keys form the primary key, or a surrogate key with the others as foreign keys.	4-19
Supertype/subtype	One relation for the supertype with shared attributes, one per subtype with the same primary key and only its own attributes.	4-20, 4-21

5

Normalization

Functional dependencies, determinants, and candidate keys. First, second, and third normal form. The Pine Valley invoice taken apart, step by step.

What normalization is

- **Normalization:** the process of successively reducing relations with anomalies to produce smaller, well-structured relations.
- Use it during logical design to check the relations you got from the seven steps, and when reverse-engineering old systems.
- Goals: less redundancy, easier referential integrity, easier maintenance, a design that mirrors the real world.
- **Normal form:** a state of a relation in which certain rules about the relationships among its attributes hold. Tonight: first through third. The rest live in Appendix B.
- Normalization is about business rules, not about how data will be used or stored. Performance is a Chapter 8 question.

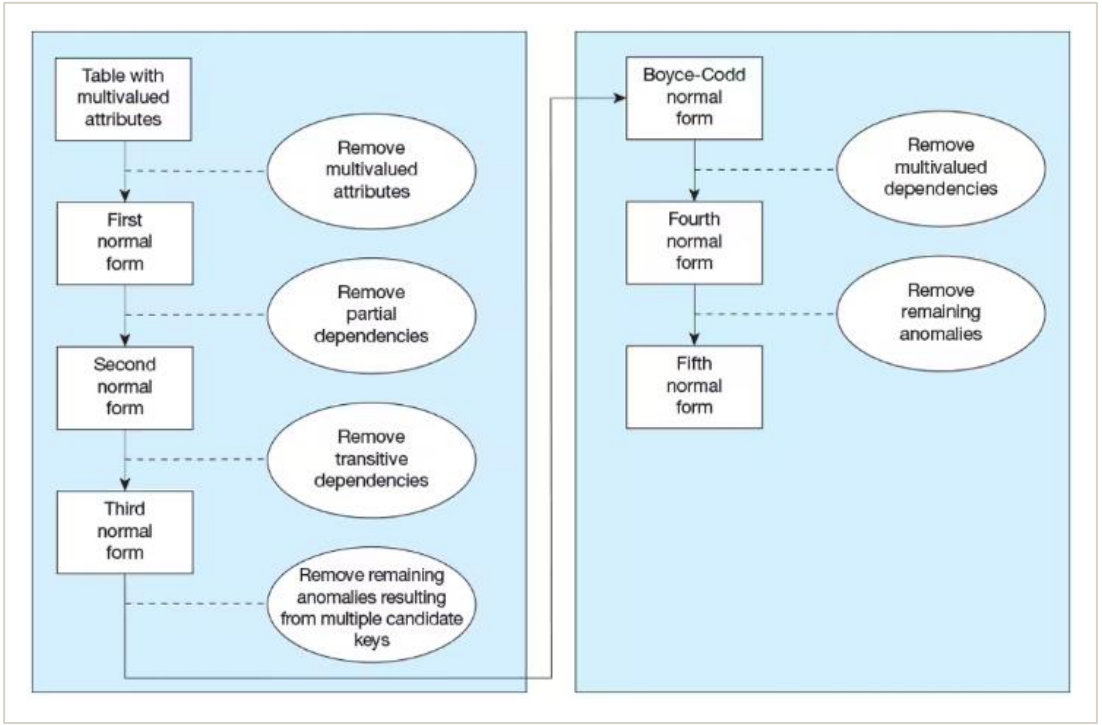


Figure 4-22. Steps in normalization (Hoffer et al., 2026)

Functional dependencies and determinants

- **Functional dependency:** for every valid value of A, that value determines exactly one value of B. Written A → B. A is the determinant.
- Not a formula. B is not computed from A. Knowing A just pins B down to one value.
- SSN → Name, Address, Birthdate. VIN → Make, Model, OrigColor. ISBN → Title, FirstAuthorName, Publisher.
- The left side can be a combination: EmpID, CourseTitle → DateCompleted. The comma means and.
- Dependencies come from business rules. Sample data can show a violation, never prove a dependency.

Your WSU ID determines your name. Your name does not determine your WSU ID, because two students can share a name.

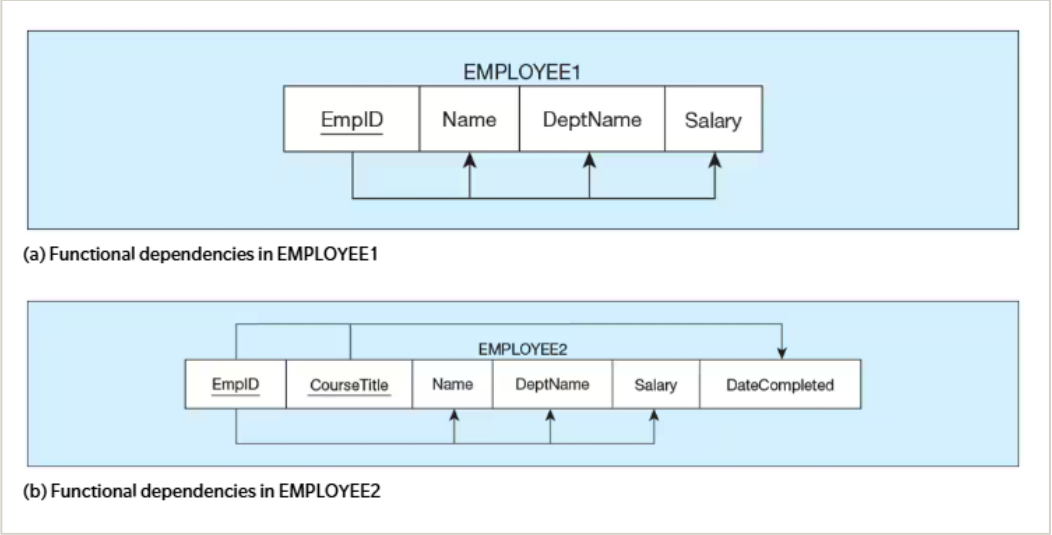


Figure 4-23. Representing functional dependencies (Hoffer et al., 2026)

Candidate keys

A candidate key is an attribute, or combination, that uniquely identifies a row. It must have two properties:

- **Unique identification.** The key's value identifies each row, so every nonkey attribute depends on it.
- **Nonredundancy.** No attribute can be removed from the key without losing unique identification.

Also commonly expected: a candidate key is never null and does not change over time.

Determinants and candidate keys

- In EMPLOYEE1, EmpID is the only determinant. It is a candidate key and the primary key.
- In EMPLOYEE2, EmpID is a determinant (it determines Name, DeptName, Salary) but not a candidate key. Only EmpID plus CourseTitle identifies a row.
- **A candidate key is always a determinant.** A determinant may or may not be a candidate key.

Preview: in a normalized relation, the primary key is the determinant for every nonkey attribute, and there are no other dependencies inside the relation.

The four determinants in INVOICE

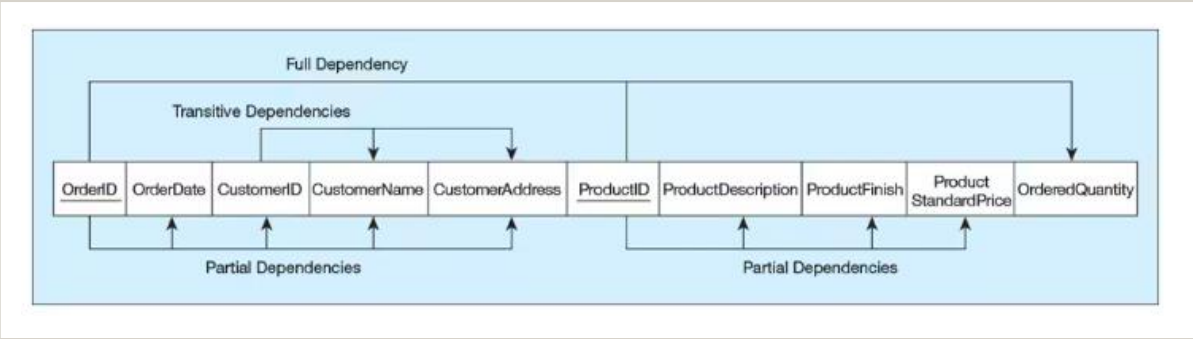


Figure 4-27. Functional dependency diagram for INVOICE (Hoffer et al., 2026)

Determinant	What it determines	Kind of dependency	Removed at
OrderID	OrderDate, CustomerID, CustomerName, CustomerAddress	Partial: only half of the key	2NF
ProductID	ProductDescription, ProductFinish, ProductStandardPrice	Partial: only half of the key	2NF
CustomerID	CustomerName, CustomerAddress	Transitive: through a nonkey attribute	3NF
OrderID, ProductID	OrderedQuantity	Full: the whole key	Stays

The only candidate key is OrderID plus ProductID. CustomerID is a determinant, but it is not part of the key, because the key already identifies every nonkey attribute.

Step 0: Put the user view in a table

PVFC Customer Invoice

Customer ID	2	Order ID	1006
Customer Name	Value Furniture	Order Date	10/24/2024
Address	15145 S.W. 17th St. Plano TX 75022		

Product ID	Product Description	Finish	Quantity	Unit Price	Extended Price
7	Dining Table	Natural Ash	2	$800.00	$1,600.00
5	Writer's Desk	Cherry	2	$325.00	$650.00
4	Entertainment Center	Natural Maple	1	$650.00	$650.00
				Total	$2,900.00

Figure 4-24. Invoice, Pine Valley Furniture Company (Hoffer et al., 2026)

OrderID	Order Date	Customer ID	Customer Name	Customer Address	ProductID	Product Description	Product Finish	Product StandardPrice	Ordered Quantity
1006	10/24/2024	2	Value Furniture	Plano, TX	7	Dining Table	Natural Ash	800.00	2
					5	Writer's Desk	Cherry	325.00	2
					4	Entertainment Center	Natural Maple	650.00	1
1007	10/25/2024	6	Furniture Gallery	Boulder, CO	11	4-Dr Dresser	Oak	500.00	4
					4	Entertainment Center	Natural Maple	650.00	3

Figure 4-25. INVOICE data (Hoffer et al., 2026)

- Write the view as one table with the attributes as columns and sample rows, repeating groups and all.
- Order 1006 lists three products under one order date and one customer. That is a repeating group, so this is not yet a relation.

Step 1: First normal form

OrderID	Order Date	Customer ID	Customer Name	Customer Address	ProductID	Product Description	Product Finish	Product StandardPrice	Ordered Quantity
1006	10/24/2024	2	Value Furniture	Plano, TX	7	Dining Table	Natural Ash	800.00	2
1006	10/24/2024	2	Value Furniture	Plano, TX	5	Writer's Desk	Cherry	325.00	2
1006	10/24/2024	2	Value Furniture	Plano, TX	4	Entertainment Center	Natural Maple	650.00	1
1007	10/25/2024	6	Furniture Gallery	Boulder, CO	11	4-Dr Dresser	Oak	500.00	4
1007	10/25/2024	6	Furniture Gallery	Boulder, CO	4	Entertainment Center	Natural Maple	650.00	3

Figure 4-26. INVOICE relation, 1NF (Hoffer et al., 2026)

```
OrderID → OrderDate, CustomerID, CustomerName, CustomerAddress
CustomerID → CustomerName, CustomerAddress
ProductID → ProductDescription, ProductFinish,
ProductStandardPrice
OrderID, ProductID → OrderedQuantity
```

A relation is in 1NF when it has no repeating groups and a primary key has been defined.

Still full of anomalies

- **Insertion:** a new product cannot be added until someone orders it. Adding a line to order 1007 means retyping the customer data.
- **Deletion:** remove the Dining Table from order 1006 and its finish and price vanish with it.
- **Modification:** a new price for the Entertainment Center must be written in every row that contains it.

Only OrderedQuantity depends on the whole key. Everything else hangs off part of the key, or off CustomerID.

Step 2: Second normal form

- **2NF:** 1NF plus no partial functional dependencies.
- **Partial dependency:** a nonkey attribute that depends on part, but not all, of the primary key. OrderID alone determines the order date. ProductID alone determines the product's price.
- For each part of the key that is a determinant, create a new relation with that part as its key and move its dependents there.
- PRODUCT and ORDERLINE are already in 3NF. CUSTOMERORDER is in 2NF but still hides a transitive dependency.
- A product can now be added before anyone orders it, and a price change happens in one row.

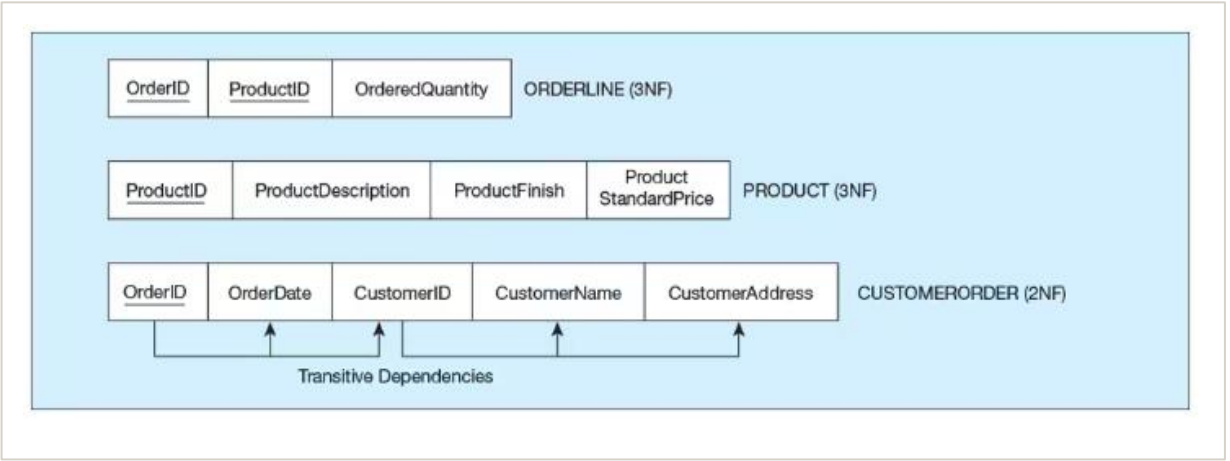


Figure 4-28. Removing partial dependencies (Hoffer et al., 2026)

Step 3: Third normal form

- **3NF:** 2NF plus no transitive dependencies.
- **Transitive dependency:** runs from the key to a nonkey attribute through another nonkey attribute. OrderID → CustomerID → CustomerName.
- For each nonkey determinant, create a new relation with it as the key, move its dependents there, and leave it behind as a foreign key so the two can still be joined.
- CUSTOMER takes CustomerName and CustomerAddress. CUSTOMERORDER becomes ORDER, with CustomerID as a foreign key.
- A customer's name and address are now entered once, not once per order.

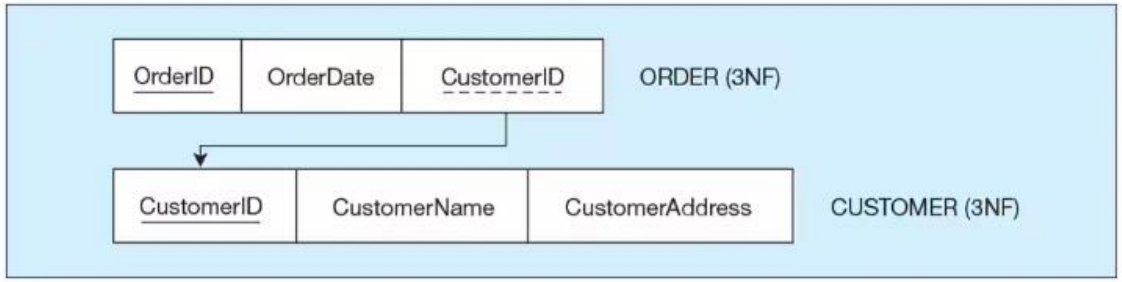


Figure 4-29. Removing transitive dependencies (Hoffer et al., 2026)

The result: four relations in third normal form

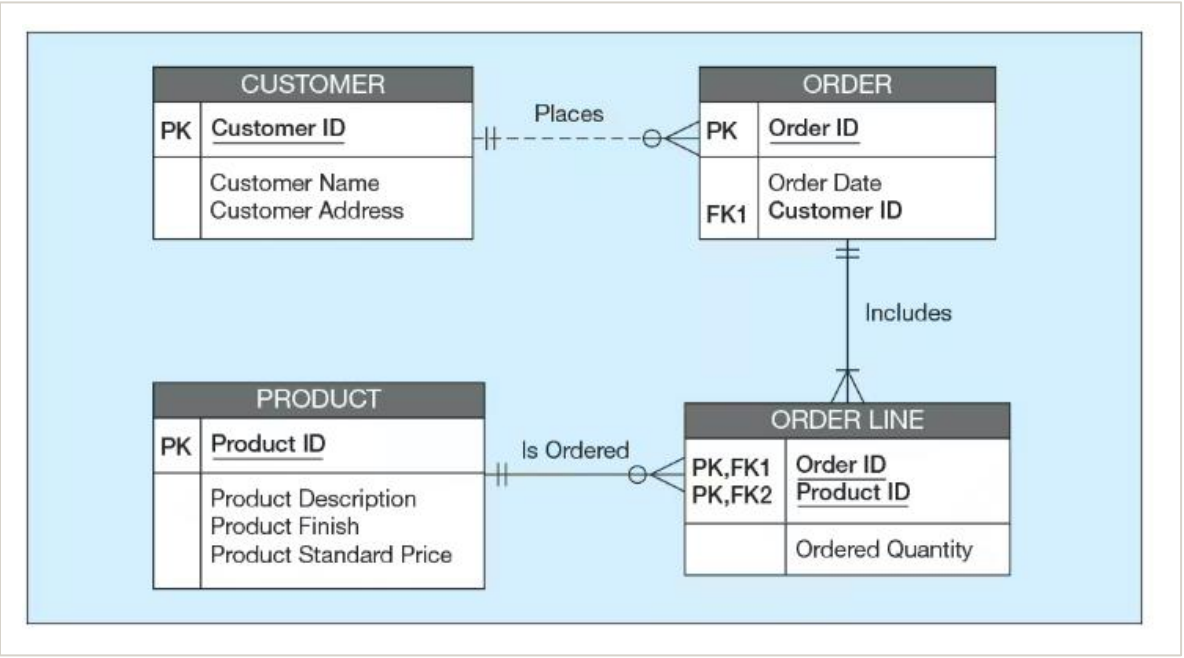


Figure 4-30. Relational schema for INVOICE data, Visio notation (Hoffer et al., 2026)

- CUSTOMER, PRODUCT, ORDER, and ORDERLINE. The same four relations as Figure 4-3, where the chapter began.
- The minimum cardinalities in the figure came from business rules, not from the data. A sample with an order for every customer would still not prove the relationship is mandatory.
- **The shortcut:** each determinant becomes the primary key of one relation, and its dependents become the nonkey attributes. Where one determinant determines another, the second stays behind as a foreign key.

Every nonkey attribute now depends on the key, the whole key, and nothing but the key. So help you Codd.

Which normal form is it in?

Not in 1NF, 1NF only, 2NF, or 3NF? Decide before the next slide.

	Relation	Functional dependencies
1	STUDENT(StudentID, StudentName, MajorName, AdvisorName)	StudentID → StudentName, MajorName, AdvisorName. MajorName → AdvisorName.
2	ENROLLMENT(StudentID, SectionID, Grade), key StudentID plus SectionID	StudentID, SectionID → Grade.
3	INVOICE(OrderID, ProductID, OrderDate, OrderedQuantity), key OrderID plus ProductID	OrderID, ProductID → OrderedQuantity. OrderID → OrderDate.
4	A table in which the CourseTitle cell for employee 100 reads "SPSS, Surveys"	No primary key has been defined.

Which normal form is it in?

Answers

	Answer	Why
1	2NF, not 3NF	The key is a single attribute, so a partial dependency is impossible. But MajorName, a nonkey attribute, determines AdvisorName. That transitive dependency keeps it out of 3NF. Split off MAJOR(MajorName, AdvisorName).
2	3NF	The only nonkey attribute depends on the whole key, and no nonkey attribute determines another.
3	1NF, not 2NF	OrderDate depends on OrderID alone, which is only part of the composite key. A partial dependency.
4	Not in 1NF	A cell holding two values is a repeating group. Give each course its own row and define a key before asking about 2NF or 3NF.

Second normal form is about part of the key. Third normal form is about a nonkey attribute determining another nonkey attribute. A relation with a single-attribute key is automatically in 2NF but can still fail 3NF.

6

Merging relations and enterprise keys

Bringing together relations from different teams and different user views without introducing new problems, and a key that never has to change.

Merging relations: four things to watch for

Relations built from different user views describe the same entities. Merging them (view integration) can go wrong in four ways.

Problem	What it is	Example	Fix
Synonyms	Different names, same meaning.	StudentID in STUDENT1 and MatriculationNo in STUDENT2 both mean the student's identity number.	Agree on one name, or choose a new one such as StudentNo. Where users need a familiar name, define an alias.
Homonyms	One name, more than one meaning.	Address in STUDENT1 is the campus address. Address in STUDENT2 is the permanent home address.	Rename to tell them apart: CampusAddress and PermanentAddress.
Transitive dependencies	Two 3NF relations that merge into one that is not.	STUDENT1(StudentID, MajorName) and STUDENT2(StudentID, Advisor) merge into STUDENT(StudentID, MajorName, Advisor). If every major has one advisor, MajorName → Advisor.	Normalize again: STUDENT(StudentID, MajorName) and MAJOR(MajorName, Advisor).
Supertype/subtype	A hidden hierarchy that looks like one entity.	PATIENT1(PatientID, Name, Address) and PATIENT2(PatientID, RoomNo). RoomNo applies only to resident patients.	Create PATIENT(PatientID, Name, Address) and RESIDENTPATIENT(PatientID, RoomNo).

Two relations with the same primary key probably describe the same entity: EMPLOYEE1(EmployeeID, Name, Address, Phone) and EMPLOYEE2(EmployeeID, Name, Address, JobCode, NoYears) merge into one EMPLOYEE relation, with Name and Address kept once.

Enterprise keys: a primary key that never changes

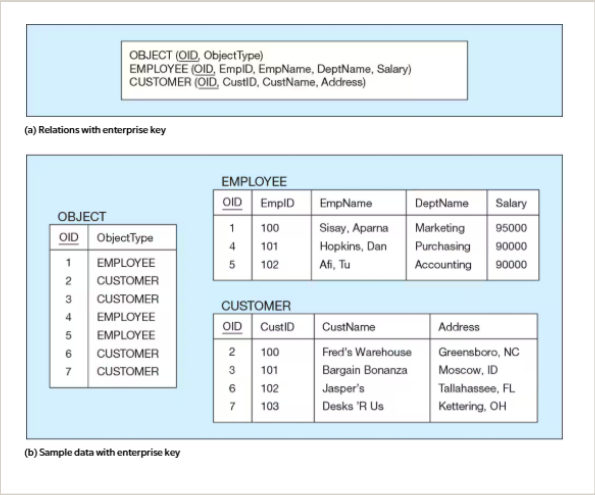


Figure 4-31 (a), (b)

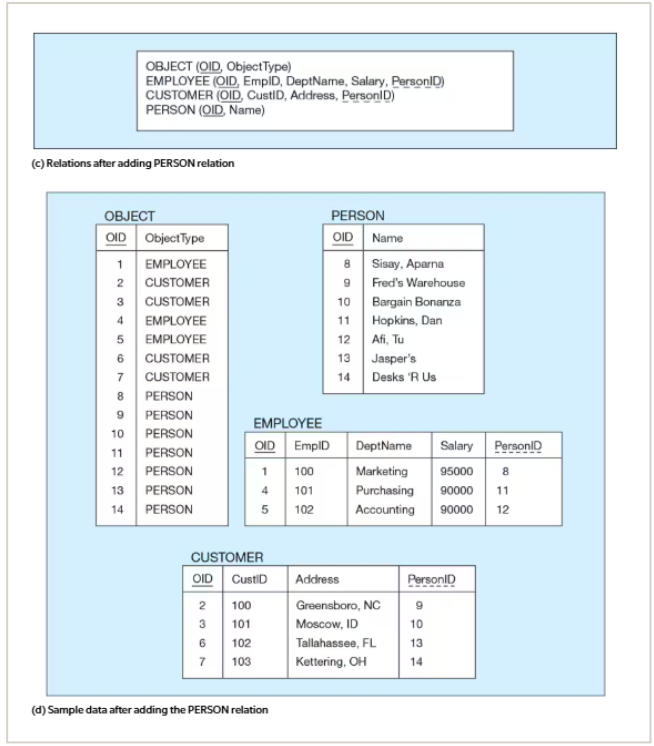


Figure 4-31 (c), (d) (Hoffer et al., 2026)

- **Enterprise key:** unique across the whole database, not just its own table, and with no business meaning.
- The natural key (EmpID, CustID) stays in the relation as nonkey data. Users still search by it.
- Why? Evolvability. When EMPLOYEE and CUSTOMER later turn out to be subtypes of PERSON, no primary key changes, so no foreign key anywhere changes.
- OBJECT is the supertype of everything, and OID is the enterprise key. Adding PERSON only adds a PersonID foreign key and moves the name.

Like giving every company document a serial number the moment it is created. Reorganize the filing later and nothing gets renumbered.

Questions?

nawaf.aljohani@wichita.edu · Woolsey Hall 308L · Ask My Instructor forum on Blackboard

Hoffer, J. A., Topi, H., Venkataraman, R., & Bala, H. (2026). Modern database management (14th ed.). Pearson.